

PROYECTO FINAL.

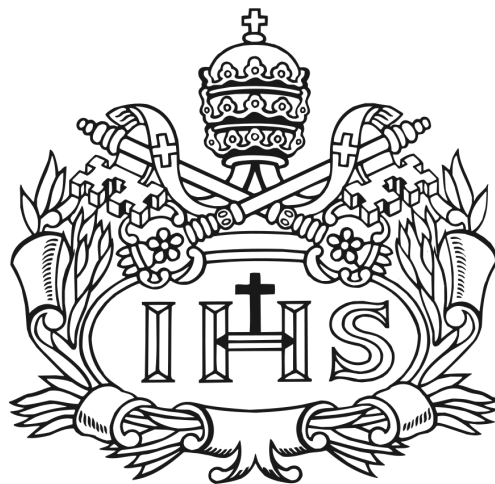
SISTEMAS OPERATIVOS .

INTEGRANTES:

ANDRES DIAZ

ALEJANDRO BELTRAN HUERTAS

MAURICIO BELTRÁN HUERTAS



Pontificia Universidad
JAVERIANA
Colombia

2025

1.Objetivos del proyecto

- Resolver un problema real mediante el uso de procesos e hilos de la biblioteca POSIX.
- Emplear mecanismos de sincronización y comunicación entre procesos usando pipes.
- Utilizar correctamente las llamadas al sistema relacionadas con la creación y manejo de hilos y procesos

2.Descripción general del proyecto

En este proyecto se implementa un sistema que permite hacer reservas por horas para entrar a un parque privado. El Parque Berlín es pequeño y durante la época de vacaciones llega mucha gente, lo que hace que el lugar se llene rápidamente y sea difícil organizar el ingreso. Por eso se creó un sistema de control de entrada basado en reservas previas.

La aplicación simula un día completo de funcionamiento del parque, donde diferentes familias intentan reservar una hora para poder entrar. Para esto se diseñó una arquitectura cliente-servidor compuesta por dos tipos de procesos:

- **Servidor que en este caso es el controlador de reserva:** es el proceso principal que administra cuántas personas pueden entrar, acepta o rechaza reservas, simula el paso del tiempo y genera reportes.
- **Cientes que en este caso son los agentes de reserva:** son procesos independientes que leen solicitudes desde archivos CSV y se las envían al servidor para que las procese.

La comunicación entre los procesos se realiza usando *pipes* nominales (FIFO), siguiendo los estándares POSIX.

2.1 Servidor: Controlador de Reserva

El servidor recibe las solicitudes enviadas por los agentes y decide si las acepta o las rechaza según el espacio disponible en el parque. También se encarga de simular el paso del tiempo: cada “hora” simulada corresponde a una cantidad configurable de segundos reales.

Cada vez que avanza una hora, el servidor realiza dos acciones:

- **Salen** las familias que entraron hace dos horas.
- **Entran** las familias que tienen reserva para la hora actual.

Además, si una reserva llega en un horario que ya está lleno, el servidor intenta **reprogramarla** buscando el siguiente bloque de tiempo disponible a partir de la hora actual.

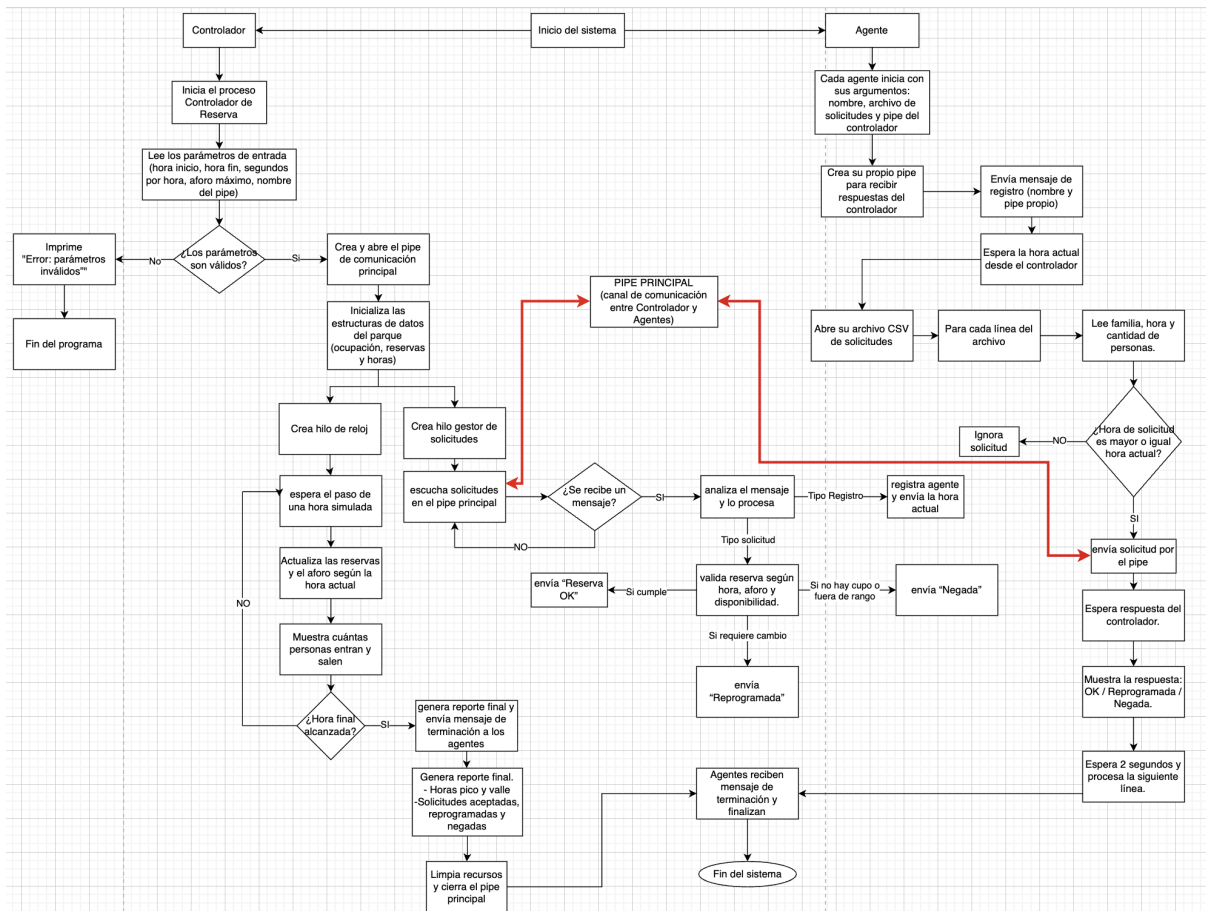
El servidor rechaza una solicitud de inmediato cuando ocurre alguna de estas condiciones:

- El número de personas supera el aforo permitido.
- La hora solicitada está por fuera del horario de funcionamiento.
- No hay espacio disponible, ni siquiera moviendo la reserva a otro horario.

2.2. Cliente: Agente de Reserva

- El cliente lee las solicitudes desde un archivo CSV con el formato: *familia, hora, personas*.
 - Ignora las solicitudes cuya hora ya pasó en relación con la hora simulada actual. Luego, envía cada solicitud al servidor y espera la respuesta correspondiente.
 - El cliente muestra en pantalla el resultado que reciba del servidor, que puede ser: *OK, REPROGRAMADA, NEGADA o NEGADA EXT*.
- Cuando termina de leer todas las solicitudes del CSV, cierra su pipe y finaliza su ejecución.

3. Arquitectura del sistema



4. Plan de pruebas

Caso de prueba	Descripción	Comando(s) ejecutados	Resultado esperado	Resultado obtenido
Prueba-01	Ejecución solo del controlador	./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador	El controlador arranca, crea el pipe y muestra avance de horas, pero no procesa solicitudes.	El controlador arranca, crea el pipe y muestra avance de horas, pero no procesa solicitudes.
Prueba-02	Ejecución del controlador + un agente con archivo solicitudes1.csv	1) ./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador2) ./agente -s Agente1 -a solicitudes1.csv -p pipeControlador	El agente se registra, se envían solicitudes, el controlador acepta, reprograma o rechaza según aforo.	El agente se registra, se envían solicitudes, el controlador acepta, reprograma o rechaza según aforo.
Prueba-03	Ejecución del controlador + mismo agente pero con archivo solicitudes2.csv	1) ./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador2) ./agente -s Agente1 -a solicitudes2.csv -p pipeControlador	El sistema debe procesar correctamente un archivo distinto, manteniendo consistencia en respuestas.	El sistema debe procesar correctamente un archivo distinto, manteniendo consistencia en respuestas.
Prueba-04	Ejecución del controlador + dos agentes simultáneos	1) ./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador2) ./agente -s Agente1 -a solicitudes1.csv -p pipeControlador3) ./agente -s Agente2 -a solicitudes2.csv -p pipeControlador	Ambos agentes deben registrarse, enviar solicitudes y recibir respuestas sin interferencia. El controlador debe manejar la concurrencia correctamente.	Ambos agentes deben registrarse, enviar solicitudes y recibir respuestas sin interferencia. El controlador debe manejar la concurrencia correctamente.
Prueba-05	Ejecución del controlador + tres agentes simultáneos	1) ./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador2) ./agente -s Agente1 -a solicitudes1.csv -p pipeControlador3) ./agente -s Agente2 -a solicitudes2.csv -p pipeControlador4) ./agente -s Agente3 -a solicitudes3.csv -p pipeControlador	El controlador debe procesar múltiples agentes concurrentes, sin bloqueos ni pérdida de mensajes.	El controlador debe procesar múltiples agentes concurrentes, sin bloqueos ni pérdida de mensajes.

Con el fin de validar el correcto funcionamiento del programa se plantearon 5 pruebas cada una con un fin específico. Asimismo, se documentaron la situación que se estaba ejecutando los comandos utilizados para ejecutar, el resultado obtenido y el resultado esperado. Cabe mencionar, que para cada una de estas pruebas se añade pantallazos de ejecución como evidencia y el objetivo de su realización.

Prueba-01:

Con el fin de verificar que el controlador inicializa correctamente (incluyendo el pipe principal, variables internas y simulación de tiempo), incluso cuando no existen agentes conectados.

Evidencia:

```

[Andres@MacBook-Air-de-JOSE Proyectofinal % make
gcc -Wall -Wextra -pthread -o controlador controlador.c controlador_funciones.c
gcc -Wall -Wextra -pthread -o agente agente.c agente_funciones.c
[Andres@MacBook-Air-de-JOSE Proyectofinal % ./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador

```

Prueba-02:

Verificar la comunicación básica entre un agente y el controlador, confirmando que el envío de solicitudes, recepción de respuestas y la gestión del aforo se realizan correctamente.

Evidencia:

```
gpc -Wall -Wextra -pthread -o agente agente.c agente.funciones.c
Andres@MacBook-Air-de-JOSE ProyectoFinal % ./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador
Recibida solicitud de Agente1: familia Zuluaga, hora 8, 10 personas
Hora actual: 7
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Recibida solicitud de Agente1: familia Dominguez, hora 8, 4 personas
Hora actual: 8
Salen: 0 personas (ninguna)
Entran: 10 personas (Zuluaga)
Recibida solicitud de Agente1: familia Rojas, hora 10, 10 personas
Recibida solicitud de Agente1: familia Perez, hora 12, 15 personas
Hora actual: 9
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Hora actual: 10
Salen: 10 personas (Zuluaga)
Entran: 4 personas (Dominguez)
Hora actual: 11
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Hora actual: 12
Salen: 4 personas (Dominguez)
Entran: 0 personas (ninguna)
Horas pico: 8 9 (10 personas)
Horas valle: 7 12 14 15 16 17 18 19 (0 personas)
Solicitudes negadas: 2
Solicitudes aceptadas en su hora: 1
Solicitudes re-programadas: 1
Andres@MacBook-Air-de-JOSE ProyectoFinal %
Andres@MacBook-Air-de-JOSE ProyectoFinal %

Last login: Sun Nov 16 14:00:15 on tty000
[Andres@MacBook-Air-de-JOSE ~ % cd Documents
[Andres@MacBook-Air-de-JOSE Documents % cd U
[Andres@MacBook-Air-de-JOSE U % cd CuartoSemestre
[Andres@MacBook-Air-de-JOSE CuartoSemestre % cd Operativos
[Andres@MacBook-Air-de-JOSE Operativos % cd ProyectoFinal
[Andres@MacBook-Air-de-JOSE ProyectoFinal % ls
agente          controlador.c      s1.csv
agente.c        controlador_funciones.c s2.csv
agente_funciones.c controlador_funciones.h solicitudes1.csv
agente_funciones.h makefile           solicitudes2.csv
controlador      pipeControlador    solicitudes3.csv
[Andres@MacBook-Air-de-JOSE ProyectoFinal % make
make: Nothing to be done for 'all'.
[Andres@MacBook-Air-de-JOSE ProyectoFinal % ./agente -s Agente1 -a solicitudes1.csv -p pipeControlador
Agente Agente1 registrado. Hora actual: 7
Respuesta: OK|Zuluaga|8
Respuesta: REPROGRAMADA|Dominguez|10
Respuesta: NEGADA|Rojas
Agente Agente1 termina.
Andres@MacBook-Air-de-JOSE ProyectoFinal %
```

Prueba-03:

Evaluar la flexibilidad y consistencia del sistema frente a diferentes fuentes de entrada, comprobando que el controlador procesa correctamente un archivo distinto.

Evidencia:

```
Andres@MacBook-Air-de-JOSE ProyectoFinal % ./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador
Recibida solicitud de Agente1: familia Garcia, hora 9, 8 personas
Hora actual: 7
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Recibida solicitud de Agente1: familia Lopez, hora 11, 12 personas
Hora actual: 8
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Recibida solicitud de Agente1: familia Martinez, hora 7, 5 personas
Hora actual: 9
Salen: 0 personas (ninguna)
Entran: 8 personas (Garcia)
Hora actual: 10
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Hora actual: 11
Salen: 8 personas (Garcia)
Entran: 5 personas (Martinez)
Hora actual: 12
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Horas pico: 11 12 (10 personas)
Horas valle: 7 8 13 14 15 16 17 18 19 (0 personas)
Solicitudes negadas: 1
Solicitudes aceptadas en su hora: 1
Solicitudes re-programadas: 1
Andres@MacBook-Air-de-JOSE ProyectoFinal %

[Andres@MacBook-Air-de-JOSE CuartoSemestre % cd Operativos
[Andres@MacBook-Air-de-JOSE Operativos % cd ProyectoFinal
[Andres@MacBook-Air-de-JOSE ProyectoFinal % ls
agente          controlador.c      s1.csv
agente.c        controlador_funciones.c s2.csv
agente_funciones.c controlador_funciones.h solicitudes1.csv
agente_funciones.h makefile           solicitudes2.csv
controlador      pipeControlador    solicitudes3.csv
[Andres@MacBook-Air-de-JOSE ProyectoFinal % make
make: Nothing to be done for 'all'.
[Andres@MacBook-Air-de-JOSE ProyectoFinal % ./agente -s Agente1 -a solicitudes1.csv -p pipeControlador
Agente Agente1 registrado. Hora actual: 7
Respuesta: OK|Zuluaga|8
Respuesta: REPROGRAMADA|Dominguez|10
Respuesta: NEGADA|Rojas
Agente Agente1 termina.
[Andres@MacBook-Air-de-JOSE ProyectoFinal % ./agente -s Agente1 -a solicitudes2.csv -p pipeControlador
Agente Agente1 registrado. Hora actual: 7
Respuesta: OK|Garcia|9
Respuesta: NEGADA|Lopez
Agente Agente1 termina.
Andres@MacBook-Air-de-JOSE ProyectoFinal %
```

Prueba-04:

Validar la correcta sincronización y concurrencia entre múltiples agentes, asegurando que los mensajes no se pierdan, no existan colisiones en el pipe y que el controlador mantenga control del aforo global con precisión.

Evidencia:

```
[Andres@MacBook-Air-de-JOSE proyectofinal % ./agente -s Agente2 -a solicitudes2.csv -p pipeControlador
Agente Agente2 registrado. Hora actual: 7
Respuesta: REPROGRAMADA|Garcia|10
Respuesta: NEGADA|Lopez
Agente Agente2 termina.
Andres@MacBook-Air-de-JOSE proyectofinal %

[Andres@MacBook-Air-de-JOSE proyectofinal % ./agente -s Agente2 -a solicitudes2.csv -p pipeControlador
Agente Agente2 registrado. Hora actual: 7
Respuesta: REPROGRAMADA|Garcia|10
Respuesta: NEGADA|Lopez
Agente Agente2 termina.
Andres@MacBook-Air-de-JOSE proyectofinal %
```

```

Andres@MacBook-Air-de-JOSE Proyectofinal % ./controlador -i 7 -f 12 -s 2 -t 10 -p pipeControlador
Recibida solicitud de Agente1: familia Zuluaga, hora 8, 10 personas
Recibida solicitud de Agente2: familia Garcia, hora 9, 8 personas
Hora actual: 7
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Recibida solicitud de Agente1: familia Dominguez, hora 8, 4 personas
Recibida solicitud de Agente2: familia Lopez, hora 11, 12 personas
Hora actual: 8
Salen: 0 personas (ninguna)
Entran: 10 personas (Zuluaga)
Recibida solicitud de Agente1: familia Rojas, hora 10, 10 personas
Recibida solicitud de Agente2: familia Martinez, hora 7, 5 personas
Hora actual: 9
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Recibida solicitud de Agente1: familia Perez, hora 12, 15 personas
Hora actual: 10
Salen: 10 personas (Zuluaga)
Entran: 8 personas (Garcia)
Hora actual: 11
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Hora actual: 12
Salen: 8 personas (Garcia)
Entran: 0 personas (ninguna)
Horas pico: 8 9 (10 personas)
Horas valle: 7 12 13 14 15 16 17 18 19 (0 personas)
Solicitudes negadas: 5
Solicitudes aceptadas en su hora: 1
Solicitudes re-programadas: 1
Andres@MacBook-Air-de-JOSE Proyectofinal %

```

Prueba-05:

Validar la correcta sincronización y concurrencia entre múltiples agentes, asegurando que los mensajes no se pierdan, no existan colisiones en el pipe y que el controlador mantenga control del aforo global con precisión. En una situación de mayor carga al utilizar 3 agentes.

Evidencia:

```

Modulo --zsh -- 80x24
(base) mauriciobeltranhurtas@MacBook-Pro-de-Mauricio-2 Modulo % ./controlador -i 7 -f 19 -s 2 -t 20 -p pipeCONTROLADOR
Recibida solicitud de A1: familia Zuluaga, hora 8, 10 personas
Recibida solicitud de A2: familia Garcia, hora 9, 8 personas
Hora actual: 7
Salen: 0 personas (ninguna)
Entran: 0 personas (ninguna)
Recibida solicitud de A1: familia Dominguez, hora 8, 4 personas
Recibida solicitud de A3: familia Hernandez, hora 8, 6 personas
Recibida solicitud de A2: familia Lopez, hora 11, 12 personas
Hora actual: 8
Salen: 0 personas (ninguna)
Entran: 10 personas (Zuluaga)
Recibida solicitud de A1: familia Rojas, hora 10, 10 personas
Recibida solicitud de A3: familia Castro, hora 9, 3 personas
Recibida solicitud de A2: familia Martinez, hora 7, 5 personas
Hora actual: 9
Salen: 0 personas (ninguna)
Entran: 8 personas (Garcia)
Recibida solicitud de A1: familia Perez, hora 12, 15 personas
Recibida solicitud de A3: familia Bermudez, hora 14, 7 personas
Hora actual: 10
Salen: 10 personas (Zuluaga)
Entran: 10 personas (Dominguez, Hernandez)

Modulo --zsh -- 80x24
(base) mauriciobeltranhurtas@MacBook-Pro-de-Mauricio-2 Modulo % ./agente -s A1 -a solicitudes1.csv -p pipeCONTROLADOR
Agente A1 registrado. Hora actual: 7
Respuesta: OK|Zuluaga|8
Respuesta: REPROGRAMADA|Dominguez|10
Respuesta: REPROGRAMADA|Rojas|14
Agente A1 termina.
(base) mauriciobeltranhurtas@MacBook-Pro-de-Mauricio-2 Modulo %

Modulo --zsh -- 80x24
(base) mauriciobeltranhurtas@MacBook-Pro-de-Mauricio-2 Modulo % ./agente -s A2 -a solicitudes2.csv -p pipeCONTROLADOR
Agente A2 registrado. Hora actual: 7
Respuesta: OK|Garcia|9
Respuesta: REPROGRAMADA|Lopez|12
Agente A2 termina.
(base) mauriciobeltranhurtas@MacBook-Pro-de-Mauricio-2 Modulo %

Modulo --zsh -- 80x24
(base) mauriciobeltranhurtas@MacBook-Pro-de-Mauricio-2 Modulo % ./agente -s A3 -a solicitudes3.csv -p pipeCONTROLADOR
Agente A3 registrado. Hora actual: 8
Respuesta: REPROGRAMADA|Hernandez|10
Respuesta: REPROGRAMADA|Castro|11
Respuesta: OK|Bermudez|14
Agente A3 termina.
(base) mauriciobeltranhurtas@MacBook-Pro-de-Mauricio-2 Modulo %

```

Datos de los archivos de entrada utilizados:

```
solicitudes1.csv  
  
# solicitudes1.csv  
  
cat > s1.csv << 'EOF'  
  
Zuluaga,8,10  
  
Dominguez,8,4  
  
Rojas,10,10  
  
Perez,12,15  
  
EOF
```

```
solicitudes2.csv  
  
# solicitudes2.csv  
  
cat > s2.csv << 'EOF'  
  
Garcia,9,8  
  
Lopez,11,12  
  
Martinez,7,5  
  
EOF
```

5. Conclusiones

El programa logra simular un sistema de reservas para un parque, considerando reservas aceptadas, negadas y re programadas, considerando el horario de este, la duración de cada una de las reservas y el aforo del parque.

De igual manera, se aplicaron mecanismos de sincronización como mutex para el buen funcionamiento del programa, factor fundamental ya que en este simulador existe un controlador y uno o más agentes cada uno con sus propias reservas y enviando sus datos. Es decir, todas las entidades funcionan correctamente sin bloquearse las unas a las otras o producir un overhead, distribuyendo apropiadamente los recursos y permitiendo la simulación.

Adicionalmente, se implementaron distintos hilos con funciones como revisar el contenido de los pipes o avanzar el reloj esta distribución de funciones permite tener un programa más ordenado y sincronizado. Finalmente, el código tiene una estructura modular que permite ubicar las funciones, variables que este maneja y su funcionamiento, sumado a los comentarios presentes en cada uno de los archivos se vuelve aún más claro y fácil de replicar.